

Digital Design with HDL

Lecture 6



Electronic and Communication Engineering
Department

Digital Systems Design Lab

Salam Kadhim

salam.alshemmari@uokufa.edu.iq

User-Defined Types

- Pre-defined types are just some of *many possible data types* in VHDL. Others can be *user-defined types*
- Syntax:

```
type type_name is [type definition];  
signal signal_name: type_name;
```

- For synthesis, all types should have an obvious mapping to a binary representation
 - User-defined types:

```
type day_of_the_month is range 0 to 31;  
signal current_day: day_of_the_month;
```

- Synthesis will convert this to binary 0 to binary 31 (5 bit bus)

- User-defined *enumeration* types:

```
type my_door is (open, closed, locked);  
signal current_state: my_door;
```

Keep in
mind!

- Synthesis will convert this to binary 0 to binary 2 (2 bit bus)
- Any operator applied to user-defined types must be defined whenever it involves different types

```
my_door <= my_door + 1; -- UNDEFINED OPERATION
```

Arrays

- An **array** in VHDL is a type that represents collection of objects of the same type. Its declaration must include the size and the type of object:

```
type myarray is array (31 downto 0) of integer;  
signal array1: myarray;
```

- Arrays are extremely useful constructs. Some of the standard and IEEE types are arrays (**string**, **bit_vector**, **std logic array**, **signed**, **unsigned**). These types are defined in the respective packages and do not need to be re-declared. For example:

```
signal array1: STD_LOGIC_VECTOR (31 downto 0);
```

is equivalent to

```
type my_vector_type is array (31 downto 0) of STD_LOGIC;  
signal my_vector : my_vector_type;
```

- Note that in hardware only one-dimensional arrays make any sense! (with a couple of exceptions)

Arrays

- *Arrays can be declared in either direction and with any finite bound:*

```
input array1: in STD_LOGIC_VECTOR (31 downto 0);  
signal array2: STD_LOGIC_VECTOR (1 to 3);
```

as long as the bounds make sense:

```
input array1: in STD_LOGIC_VECTOR (0 downto 6); --ILLEGAL  
signal array2: STD_LOGIC_VECTOR (3 to 0); --ILLEGAL
```

*For synthesizable VHDL, the use of **to** is strongly discouraged!*

- *Any “slice” of an array can be assigned, including a single element, as long as the two sides have the same size:*

```
array2(1) <= '0';  
array2 <= array1(2 downto 0);  
array1(15 downto 8) <= "01010101";
```

- *Once a direction has been declared, it cannot be changed:*

```
array1(0 to 7) <= "01010101"; --ILLEGAL
```

Arrays

```
signal array1: STD_LOGIC_VECTOR (4 downto 0);  
signal array2: STD_LOGIC_VECTOR (2 downto 0);  
signal a,b,c: STD_LOGIC;  
signal array3: STD_LOGIC_VECTOR (1 downto 0);
```

- Arrays can be defined with the **concatenation operator &**:

```
array2 <= a & b & c;           -- order is important (downto)  
array1 <= array3 & array2;    -- order is important (downto)
```

- In comparison operators, array operands can be of different lengths, but are left-aligned:

```
array1 <= "10011";  
array2 <= "111";  
if (array2 > array 1) then [...]
```

Returns TRUE for unsigned numbers! **So don't do it!**

Arrays: memories

*An exception to the one-dimension rule is in the definition of **memories** (both ROMs and RAMs).*

```
entity async_rom16x8 is
    Port ( Address : in  UNSIGNED (3 downto 0);
          DataOut  : out STD_LOGIC_VECTOR (7 downto 0));
end async_rom16x8;

architecture arch of async_rom16x8 is

    type ROM_Array is array (0 to 15) of std_logic_vector(7 downto 0);
    constant Content: ROM_Array := (
        0 => B"00100100",
        1 => B"00000001",
        2 => B"00010010",
        5 => B"11111111",
        6 => B"00000011",
        others => B"00000000");

begin
    DataOut <= Content(to_integer(Address));
end arch;
```

Size of ROM (16x8)

Contents of addresses 0, 1, 2, 5, 6
(B"XXXX") = binary number

0s for all other addresses

Library function

Asynchronous read (no clock)

Arrays: memories

```
entity sync_rom16x8 is
    Port (CLK, REN : in  STD_LOGIC;
          Address : in  UNSIGNED (3 downto 0);
          DataOut  : out  STD_LOGIC_VECTOR (7 downto 0));
end sync_rom16x8;
```

Size of ROM (16x8)

```
architecture arch of sync_rom16x8 is
    type ROM_Array is array (0 to 15) of std_logic_vector(7 downto 0);
    constant Content: ROM_Array := (
        0 => B"00100100",
        1 => B"00000001",
        2 => B"00010010",
        5 => B"11111111",
        6 => B"00000011",
        others => B"00000000");
begin
    process (CLK)
    begin
        if (rising_edge(CLK)) then
            if (REN='1') then
                DataOut <= Content(to_integer(Address));
            end if;
        end if;
    end process;
end arch;
```

Contents of addresses 0, 1, 2, 5, 6
(B"XXXX") = binary number

0s for all other addresses

Synchronous read
(rising edge)

No reset in a memory

Arrays: memories

```
entity dual_port_ram16x8 is
    Port ( CLK, WEN : in  STD_LOGIC;
           WAddress : in  UNSIGNED (3 downto 0);
           RAddress : in  UNSIGNED (3 downto 0);
           DataIn  : in  STD_LOGIC_VECTOR (7 downto 0);
           DataOut  : out STD_LOGIC_VECTOR (7 downto 0));
end dual_port_ram16x8;
```

Separate read and write ports

```
architecture arch of dual_port_ram16x8 is

    type ram_type is array (0 to 15) of std_logic_vector(7 downto 0);
    signal my_ram: ram_type := (
        0 => X"F3",
        1 => X"06",
        others => X"00");
begin

    DataOut <= tmp_ram(to_integer(RAddress));

    process (CLK)
    begin
        if (rising_edge(CLK)) then
            if (WEN='1') then
                my_ram(to_integer(WAddress)) <= DataIn;
            end if;
        end if;
    end process;
end arch;
```

Signal, not constant for values (optional, rare, initialization)

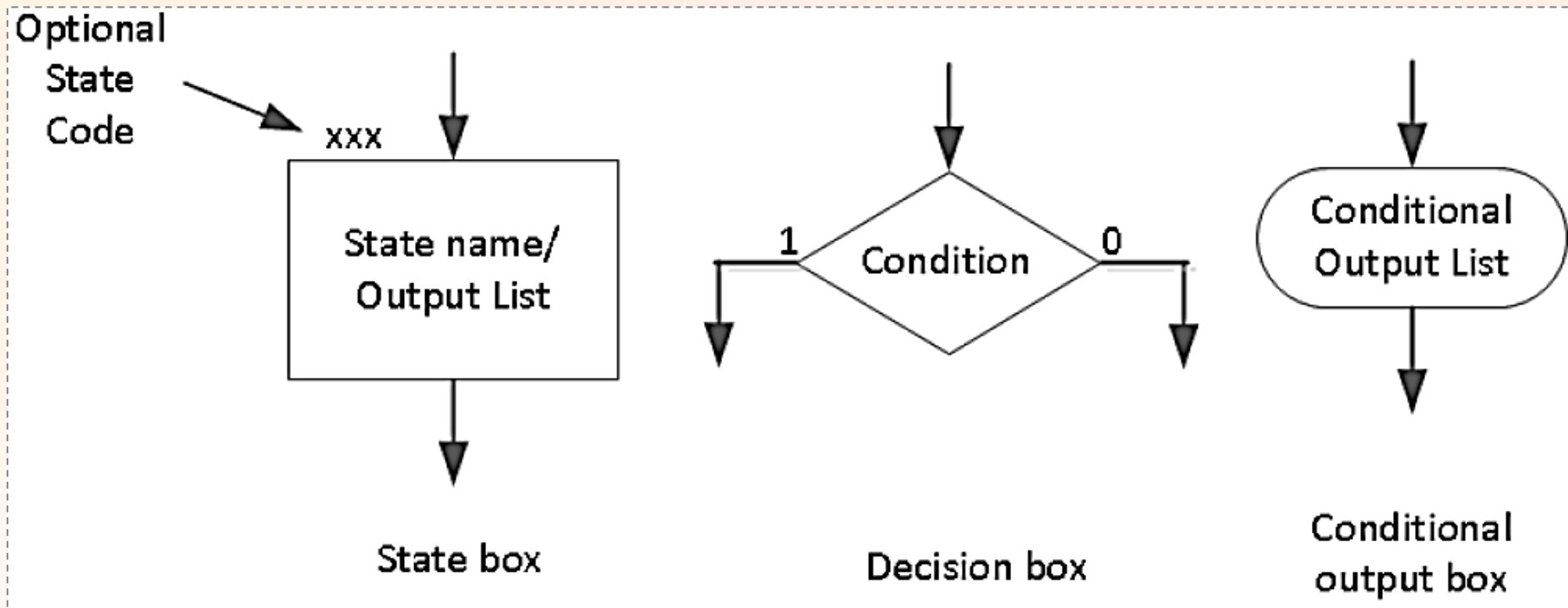
Asynchronous read

Synchronous write (rising edge)

No reset in a memory

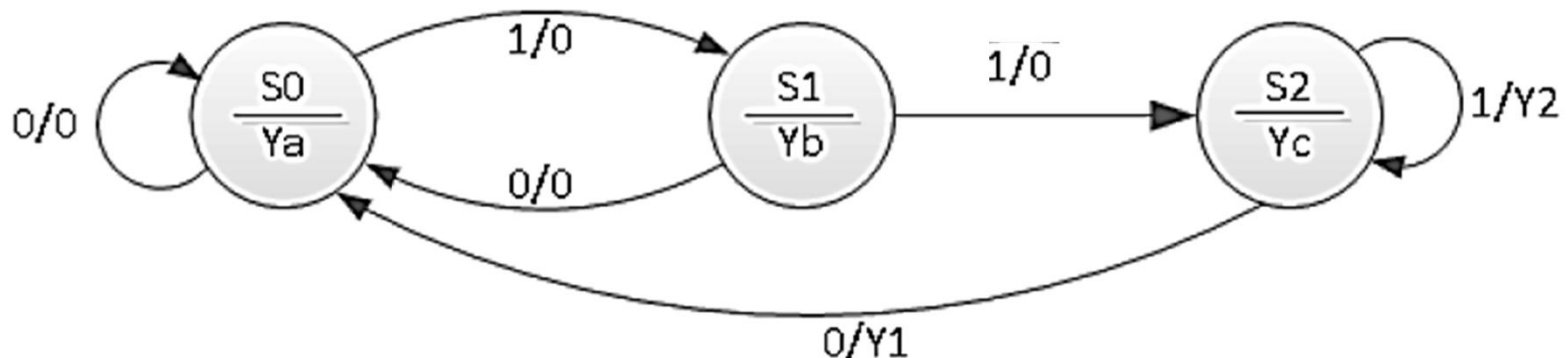
Algorithmic State Machine (ASM)

- The following diagram shows the three main components of an **ASM chart**.

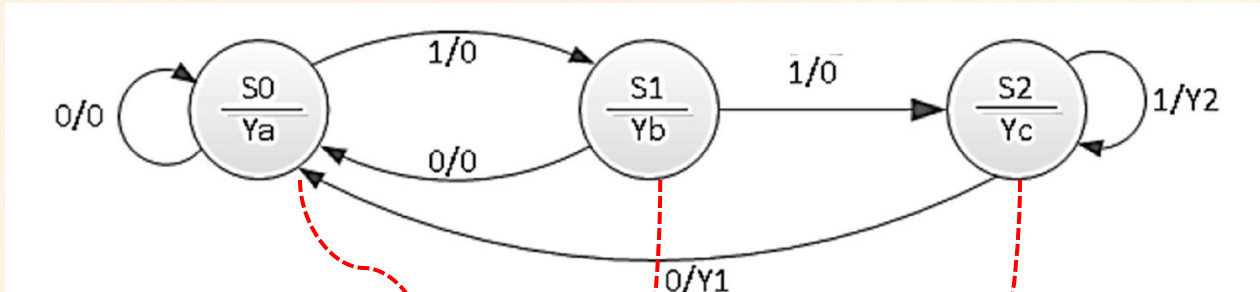


State Machine to ASM Example

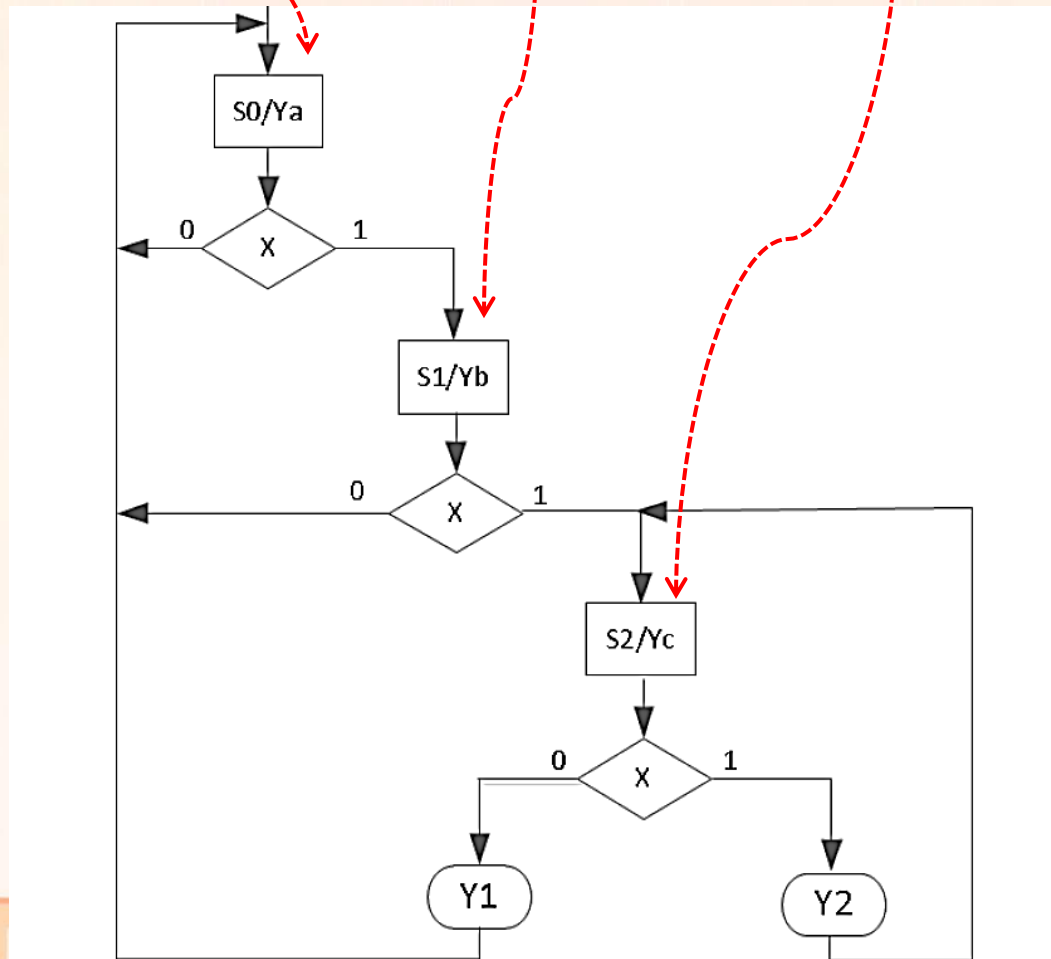
- The state graph below has both Mealy and Moore outputs.
 - Y1 and Y2 are the Mealy outputs and so should be conditional outputs.
 - Ya, Yb, and Yc are the Moore outputs so they should be part of state box. Input
- X can either be “0” or “1” and hence it should be part of the decision box.



State Machine to ASM Example



State Machine



ASM

ASM Chart to VHDL

- Once the **ASM chart** is determined, the conversion to *HDL* is straight forward.
- A **case statement** can be used to specify what happens in each state.
- Each condition box corresponds directly to **an *if statement*** (or an **else if**).
- The following code represents the functionality of the above ASM chart.

ASM to VHDL

```
type state_type is (S0, S1, S2);
signal state, next_state : state_type;
SYNC_PROC : process (clk)
begin
    if rising_edge(clk) then
        state <= next_state;
    end if;
end process;

NEXT_STATE_DECODE : process (state, x) --
    Process to compute both Mealy output &
    nextstate
begin
    y1 <= '0';
    y2 <= '0';

    case (state) is
        when S0 =>
            if (x = '1') then
                next_state <= S1;
            else
                next_state <= S0;
            end if;
        when S1 =>
            if (x = '1') then
                next_state <= S2;
            else
                next_state <= S0;
            end if;
    end case;
end process;
```

```
when S2 =>
    if (x = '1') then
        y2 <= '1';
        next_state <= S1;
    else
        y1 <= '1';
        next_state <= S0;
    end if;

when others =>
    next_state <= S0;

end case;
end process;

OUTPUT_DECODE : process (state) -- Process to
    compute Moore output
begin
    case (state) is
        when S0 => ya <= '1';
        when S1 => yb <= '1';
        when S2 => yc <= '1';
        when others =>
            ya <= '0';
            yb <= '0';
            yc <= '0';
        end case;
    end process;
```